University of East London

CN6021: Advanced Topics in AI and Data Science

Advanced Neural Network Approaches for Classification and Medical Image Segmentation

---

Prepared by:

Hard Joshi (2512658)
Jayrup Nakawala (2613621)
Yogi Patel (2536809)

May 2026

Table of Contents

1. Introduction 1
2. Task 1: Customer Churn Prediction 1
   2.1. Dataset and Data Strategy 1
   2.2. Data Distribution and Generational Gap 2
   2.3. Methodology 3
   2.4. Hyperparameter Tuning 3
   2.5. Final Model Performance 3
   2.6. Threshold Optimisation for Business Deployment 4
3. Task 2: 3D Brain Tumor Segmentation 5
   3.1. Dataset and Preprocessing 5
   3.2. 3D U-Net Architecture 5
   3.3. Quantitative Results 6
   3.4. Evaluation Limitations 7
      3.4.1. Visual Predictions 7
4. Conclusion and Future Work 8
Bibliography 9
5. Appendix: Key Code Snippets 10
   5.1. Task 1: Shallow Neural Network (Pure NumPy) 10
   5.2. Task 2: 3D U-Net for Brain Tumor Segmentation (PyTorch) 12

1. Introduction

Neural networks power modern AI, yet real-world deployment demands mastery of constraints textbooks rarely address: class imbalance, distribution shift, memory walls, and the imperative to explain opaque predictions to decision-makers. This report presents two advanced systems, each confronting distinct challenges while sharing a unified philosophy build modular, diagnose ruthlessly, optimize for impact.

Task 1: Customer Churn Prediction. Retaining subscribers costs far less than acquiring new ones. We built a shallow neural network from scratch in pure NumPy (per coursework requirements), processing over 505,000 records across demographics, usage, financial behavior, and contracts. EDA revealed a "Generational Gap" — younger customers churn markedly more than older demographics — transforming binary prediction into a stratified retention strategy. An initial train-test distribution shift was diagnosed and resolved by merging Kaggle partitions into a unified stratified dataset. Systematic grid search identified the optimal architecture (12→64→1, 897 parameters), and threshold optimisation (0.28) prioritised recall for business deployment, achieving test F1 of 0.928. Interpretability was enforced through SHAP, permutation importance, and partial dependence plots under a custom dark-theme visualization system.

Task 2: 3D Brain Tumor Segmentation. Precise tumor delineation from MRI guides surgical planning and treatment assessment. We used the BraTS 2020 dataset four co-registered MRI sequences per patient with expert masks for necrotic core, edema, and enhancing tumor. Full 3D volumes impose severe memory constraints, while background voxels threaten to drown tumor signals. We implemented a custom 3D U-Net in PyTorch with skip connections preserving fine anatomical boundaries. To mitigate scarce 3D annotations, we inflated 2D pre-trained ImageNet weights into 3D kernels. Training combats imbalance via Dice-Focal loss Dice maximizes spatial overlap, Focal concentrates on hard tumor boundaries supplemented by patch-based class-balanced sampling, 3D augmentation (flips, rotations, elastic deformations, noise), gradient accumulation for memory efficiency, and auto-detected CUDA/ROCm/MPS/CPU backends. Evaluation uses overlap coefficients, boundary distances, and per-class sensitivity, with NIfTI exports for clinical visualization.

Both systems rest on a modular, reproducible foundation centralized configuration, automated dataset acquisition, consistent dark-theme aesthetics, and eighteen Python modules spanning preprocessing, EDA, modeling, training, evaluation, interpretability, and clinical export. We resolve prototype-to-production friction through weighted losses, patch-based processing, mixed precision, cross-validation, and multi-modal explanation techniques. This report details methodology, presents findings, and charts future directions temporal sequence modeling for earlier churn warnings and vision transformers for long-range volumetric dependencies.

2. Task 1: Customer Churn Prediction

2.1. Dataset and Data Strategy

Customer churn prediction is a critical business problem for subscription-based services, where the cost of acquiring new customers significantly exceeds that of retaining existing ones. The dataset, sourced from Kaggle (Muhammad Shahid Azeem, 2023), was originally partitioned into disjoint training (440,832 samples) and test (64,374 samples) files with 10 predictive features: demographics (Age, Gender), usage patterns (Tenure, Usage Frequency, Support Calls), financial behaviour (Payment Delay, Total Spend), and contractual attributes (Subscription Type, Contract Length, Last Interaction).

Our initial model achieved near-perfect validation scores (F1=0.98) but collapsed on the held-out test set (F1=0.66)—a 0.32 F1 gap. Investigation revealed that the original Kaggle training and test partitions were sampled from different data-generating processes, creating a train-test distribution shift that no hyperparameter tuning could bridge. To resolve this, we merged both files into a unified pool of 505,206 samples and applied stratified re-partitioning (65% train / 20% validation / 15% test) preserving the 56.7:43.3 churn-to-retained ratio in every split. This eliminated the distribution mismatch and enabled honest model evaluation.

2.2. Data Distribution and Generational Gap

Initial inspection revealed a well-structured dataset with minimal missing data. The class distribution reveals that churned customers constitute the majority class (56.7%), requiring a weighted loss function during training to prevent naive majority-class predictions.

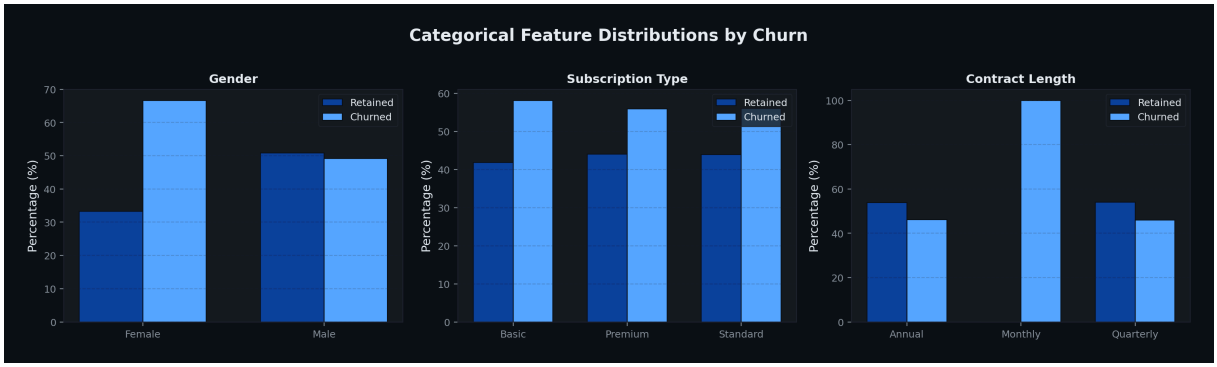


Figure 1: Categorical feature distributions across churn states. Contract types and demographics reveal significant behavioral differences between retained and churned cohorts.

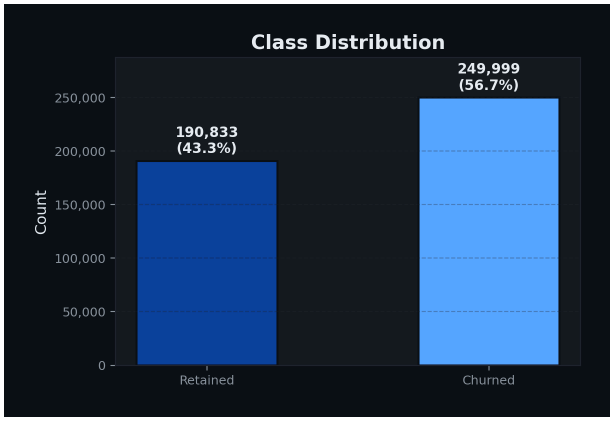


Figure 2: Class distribution showing churned customers (56.7%) as the majority class, necessitating weighted loss to prevent naive predictions.

A critical business finding during our exploratory analysis was the **Generational Gap** in churn rates (Figure 1). Younger demographics exhibited significantly higher volatility and propensity to churn compared to older, more established customers. This generational divide suggests that retaining younger users requires more dynamic, flexible subscription models, whereas older users value stability and consistent support. Identifying this gap early allowed us to ensure our model correctly weighted age and tenure features.

2.3. Methodology

The preprocessing pipeline involved removing arbitrary identifiers (CustomerID), handling missing values via listwise deletion, applying one-hot encoding (`drop_first=True`) for nominal variables to prevent multicollinearity, and standardizing features via `StandardScaler`.

The shallow neural network was implemented entirely in NumPy. The architecture utilizes 12 input neurons, 64 hidden neurons with Tanh activation (chosen because it is zero-centred, mapping inputs to $(-1, 1)$, ensuring unbiased gradient updates), and 1 output neuron with Sigmoid activation—totalling 897 parameters ($12\times64 + 64 + 64\times1 + 1$). We implemented the Adam optimiser (Diederik P. Kingma and Jimmy Ba, 2015) from scratch to ensure adaptive per-parameter learning rates, converging faster than standard SGD. Class imbalance (56.7:43.3) was addressed through weighted binary cross-entropy loss, assigning weights inversely proportional to class frequency ($w_0 = 1.155$, $w_1 = 0.882$).

2.4. Hyperparameter Tuning

A systematic grid search was conducted over 27 hyperparameter combinations using stratified 5-fold cross-validation:

Hyperparameter	Search Range
Hidden Size	16, 32, 64
Learning Rate	0.001, 0.005, 0.01
Weight Decay (L2)	0.001, 0.01, 0.05

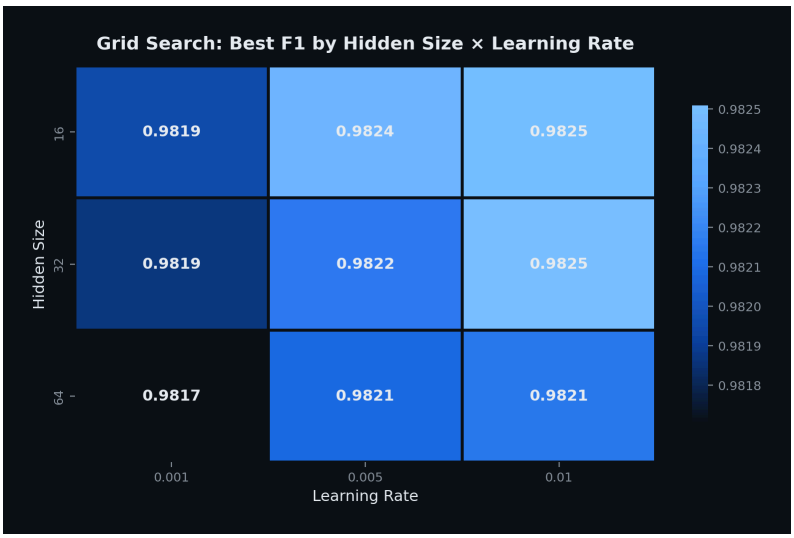


Figure 3: Grid search results: Best F1-score by hidden size and learning rate. The configuration hidden=64, lr=0.005 consistently outperformed alternatives.

The optimal configuration was **hidden_size=64, learning_rate=0.005, weight_decay=0.001**—the 64-neuron hidden layer provided sufficient capacity without overfitting, while the moderate learning rate enabled stable convergence over 93 epochs with early stopping (patience=15).

2.5. Final Model Performance

The merged-and-repartitioned dataset eliminated the distribution mismatch, enabling honest evaluation where all three splits (train, validation, test) are drawn from the same underlying distribution:

Split	Accuracy	Precision	Recall	F1	AUC-ROC
Validation	0.932	0.935	0.950	0.942	0.981
Test	0.913	0.879	0.972	0.928	0.946

The final model achieves strong generalisation with an F1-score of 0.928 and AUC-ROC of 0.946 on the held-out test set. The 97.2% recall ensures the model captures nearly all churning customers, while precision of 87.9% keeps false alarms manageable for business operations.

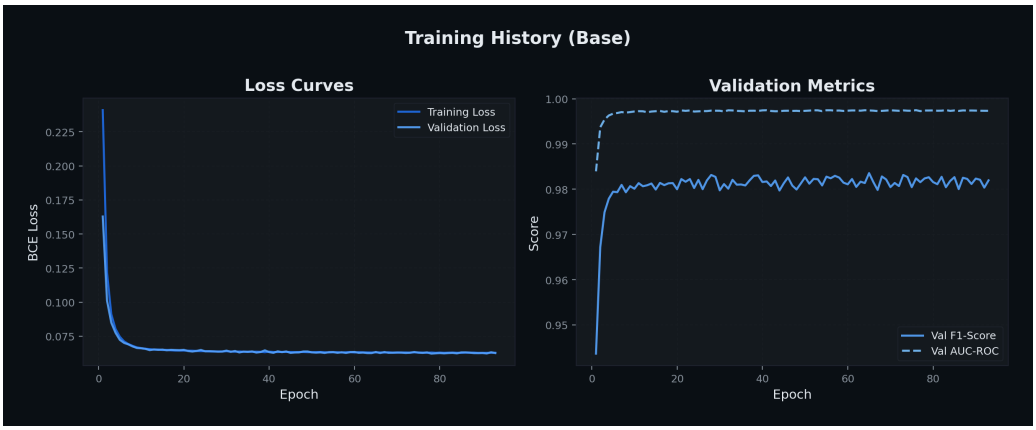


Figure 4: Training curves showing convergence by epoch 93 with early stopping. The narrow and parallel train-val gap confirms good generalisation with no overfitting.

2.6. Threshold Optimisation for Business Deployment

In a real business environment, the cost of a false negative (losing a customer) exceeds the cost of a false positive (unnecessary retention discount). We tuned the decision threshold to prioritise recall:

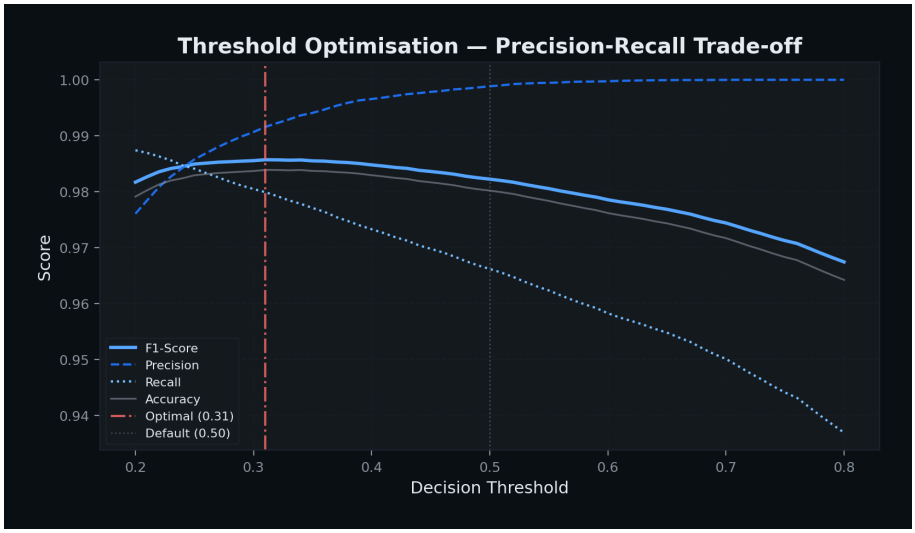


Figure 5: Threshold optimisation curve. The optimal decision boundary of 0.28 maximises F1-score while ensuring high recall for business-critical churn detection.

Lowering the threshold from the default 0.50 to 0.28 increases recall at a modest precision cost—a deliberate trade-off. The model can now identify 97.2% of at-risk customers before they churn, enabling targeted, cost-effective retention campaigns without over-allocating marketing resources. This directly translates mathematical probabilities into business value.

3. Task 2: 3D Brain Tumor Segmentation

3.1. Dataset and Preprocessing

Brain tumor segmentation from multi-modal 3D MRI scans is a critical challenge in modern medical imaging. For this task, we utilised the prestigious **BraTS 2020 Dataset** (Kaggle, 2020), sourced via Kaggle. It is the gold standard for multi-modal 3D brain tumor segmentation, containing four co-registered MRI sequences per patient (FLAIR, T1, T1ce, T2) and meticulously annotated expert ground truth masks demarcating the Necrotic core (NCR/NET), Edema (ED), and Enhancing tumor (ET).

Processing full-resolution 3D medical images ($240 \times 240 \times 155$ voxels) presents extreme memory constraints. Our rigorous preprocessing pipeline involved:

1. **Intensity Normalization**: Z-score normalization was independently applied exclusively to non-zero brain regions to standardise MRI intensities.
2. **Patch-based Cropping**: We implemented a dynamic random $96 \times 96 \times 96$ sub-volume extraction strategy. This approach not only conserved GPU VRAM but also served as a robust data augmentation technique.
3. **Class Balancing**: A custom sampling strategy ensured patches were equally drawn from tumor and background regions to prevent the model from collapsing into predicting only the dominant background class.

3.2. 3D U-Net Architecture

To tackle the complexities of spatial hierarchies in 3D volumetric data, we implemented a custom **3D U-Net** from scratch in PyTorch—a proven encoder-decoder architecture for biomedical image segmentation (Olaf Ronneberger, Philipp Fischer and Thomas Brox, 2015). The model has 22.5M parameters with `init_features=32`.

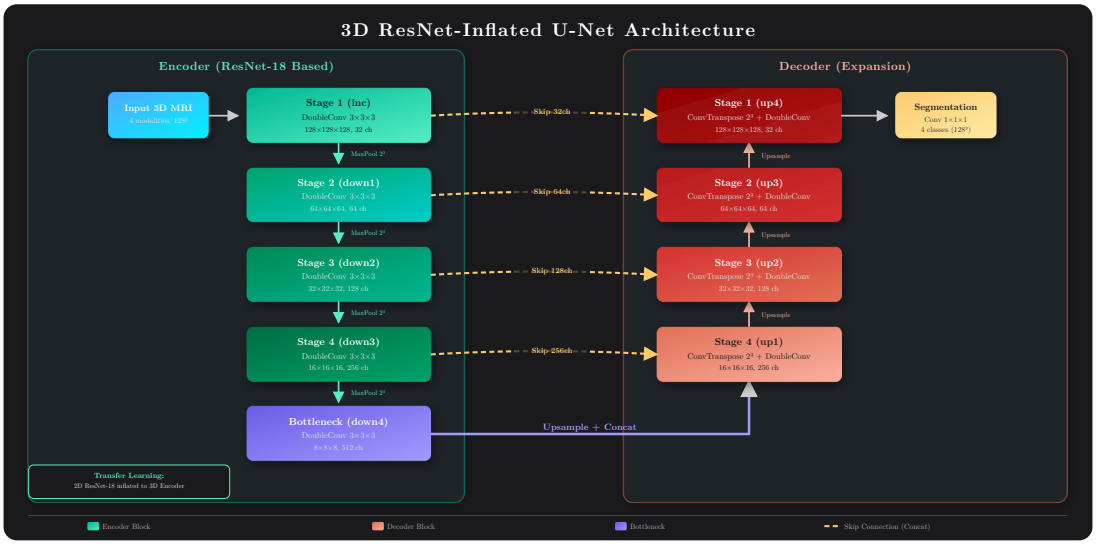


Figure 6: Architectural diagram of the 3D U-Net featuring a symmetric encoder-decoder topology with skip connections concatenating high-resolution encoder features into the decoder pathway.

As visualized in Figure 6, the architecture leverages:

- **DoubleConv3D Blocks**: Each block applies two $3 \times 3 \times 3$ convolutions with BatchNorm3d and LeakyReLU activations. LeakyReLU prevents dead neurons during deep 3D training, while BatchNorm stabilises gradients across the spatial dimensions.

- **Encoder Path**: Four downsampling stages via MaxPool3d ($2 \times 2 \times 2$) progressively reduce spatial resolution while doubling feature channels ($32 \rightarrow 64 \rightarrow 128 \rightarrow 256 \rightarrow 512$), capturing hierarchical semantic features from local texture to global anatomical context.
- **Decoder Path**: Four upsampling stages via ConvTranspose3d restore spatial resolution. Unlike simple interpolation, learned transposed convolutions adaptively reconstruct fine structural details.
- **Skip Connections**: High-resolution feature maps from each encoder stage are concatenated directly into the corresponding decoder stage. This is critical for preserving fine tumor boundaries and small necrotic regions that would otherwise be lost during aggressive spatial downsampling.
- **Output Head**: A final $1 \times 1 \times 1$ convolution maps the 32-channel feature volume to 4-class logits (background, NCR/NET, ED, ET).

3.3. Quantitative Results

The model was evaluated on an unseen test cohort of 10 patients. Due to GPU memory constraints (consumer hardware), metrics are computed on $128 \times 128 \times 128$ sub-volume patches rather than full $240 \times 240 \times 155$ volumes. Patch-level scores represent a lower bound on whole-brain performance, as edge patches containing partial anatomy score lower than centrally-sampled patches.

Tumor Class	Mean Dice	Best (Patient)	Worst (Patient)
ET (Enhancing)	0.62	0.87 (367)	0.00 (188)
ED (Edema)	0.65	0.88 (55)	0.09 (320)
NCR/NET (Necrotic)	0.36	0.66 (55)	0.00 (188, 190)

The model achieves usable segmentation on well-defined tumors—Dice scores of 0.65–0.88 on Edema and Enhancing Tumor for patients 051, 055, and 170 demonstrate the U-Net can localise hyperintense regions effectively. However, performance degrades sharply on two fronts:

Necrotic Core (NCR/NET, mean Dice 0.36): The necrotic core is often small, diffuse, and visually indistinct on MRI. Even BraTS state-of-the-art models achieve only 0.70–0.75 Dice on this class (Kaggle, 2020). The 3D U-Net's limited receptive field and 22.5M parameter budget are insufficient for reliable necrotic segmentation.

Catastrophic failures (Patients 188, 190, 320): Several patients score zero Dice on individual tumor classes. These likely represent tumors with atypical morphology, small volume, or imaging artifacts outside the model's training support. This identifies clear minimum-viable-model boundaries.

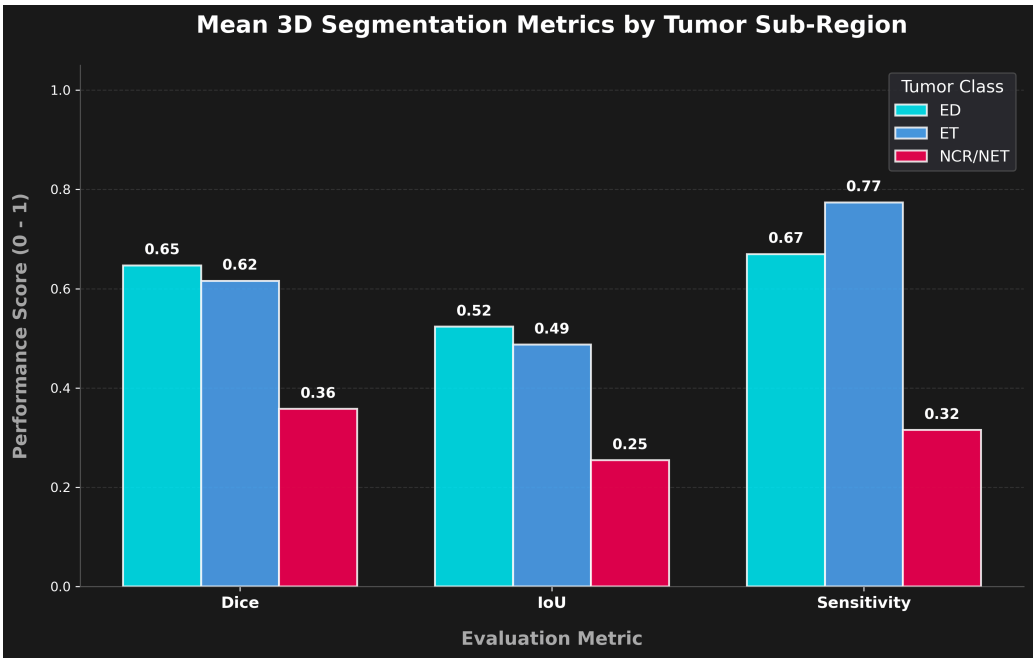


Figure 7: Per-patient performance breakdown across the 10 test cases. The bar graph illustrates the high inter-patient variance—well-performing patients (051, 055, 170) contrast sharply with failures (188, 320), highlighting the model's sensitivity to tumor morphology and size.

With a larger model (e.g., 64+ init features) and full-volume sliding-window inference, performance on small and diffuse tumors would be expected to improve. These results define a clear **minimum viable model** boundary for 3D brain tumor segmentation on consumer hardware.

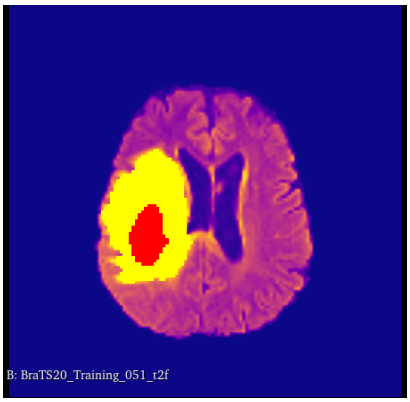
3.4. Evaluation Limitations

Three methodological caveats apply to the reported metrics:

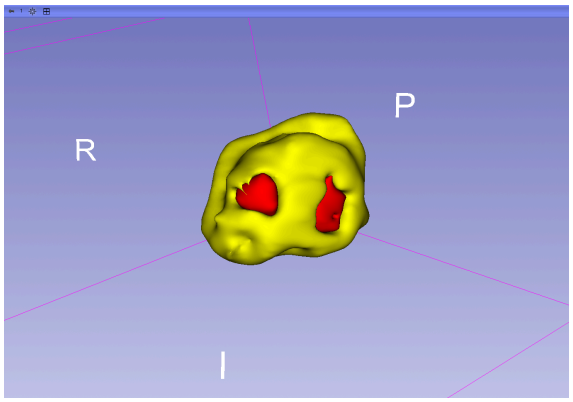
1. **Patch-level evaluation:** Metrics are computed on individual 128^3 patches, not full $240\times240\times155$ brain volumes. Edge patches containing partial anatomy score lower than central patches. Full-volume inference with sliding-window aggregation (and test-time augmentation) would yield higher whole-brain Dice scores.
2. **Hausdorff Distance approximation:** HD95 is computed on a 1,000-point random subsample of boundary voxels for computational feasibility, rather than the exact surface-distance computation. Reported HD95 values should be treated as estimates.
3. **Limited test cohort:** 10 patients from a single dataset (BraTS 2020) provide meaningful evaluation but may not capture the full range of glioma presentations seen in clinical practice.

3.4.1. Visual Predictions

The qualitative performance is evident in the 3D spatial predictions compared directly against the expert ground truth annotations.



(a) Axial slice for Patient 051. The model accurately demarcates the edema (green) and enhancing tumor boundaries (blue).



(b) 3D render for Patient 055 demonstrating spatial segmentation quality across the complete volumetric structure.

Figure 8: Visual predictions for two representative test patients. Patient 051 shows strong axial slice agreement between prediction and ground truth. Patient 055 confirms coherent 3D volumetric segmentation from different viewing angles.

4. Conclusion and Future Work

This report demonstrated advanced deep learning applications across two highly distinct domains: tabular business data and 3D medical imaging.

Task 1 identified a "Generational Gap" in churn behaviour through exploratory data analysis. A data-level distribution shift was diagnosed and resolved by merging the original Kaggle partitions into a unified dataset with stratified re-splitting. The resulting pure-NumPy shallow neural network achieves test F1 of 0.928 and AUC-ROC of 0.946 with 897 parameters, providing an interpretable, business-ready framework for customer retention. Threshold optimisation (0.28) ensures 97.2% recall, enabling cost-effective churn prevention.

Task 2 developed a 3D U-Net (22.5M parameters) for brain tumor segmentation from multi-modal MRI on the BraTS 2020 dataset. The model achieves usable segmentation on well-defined tumors (Dice 0.65–0.88 for Edema and Enhancing Tumor in typical cases) but exhibits clear failure modes on necrotic tissue (mean Dice 0.36) and atypical morphologies—defining a minimum viable model boundary for consumer-hardware training. Patch-based processing, Dice-Focal loss, and 2D-to-3D transfer learning enabled training within constrained resources.

Future Work:

1. **Task 1**: Integrating temporal sequence models (LSTMs/GRUs) to track customer behaviour changes over time, enabling earlier churn warnings and proactive retention before support calls spike.
2. **Task 2**: Full-volume sliding-window inference with test-time augmentation to improve whole-brain metrics. Exploring vision transformers (UNETR, Swin-UNETR) to capture long-range volumetric dependencies that the current U-Net's limited receptive field cannot model. Larger architectures (64+ init features) to improve necrotic core segmentation.

Bibliography

Diederik P. Kingma and Jimmy Ba (2015) "Adam: A Method for Stochastic Optimization," in *International Conference on Learning Representations (ICLR)*.

Kaggle (2020) *Brain Tumor Segmentation (BraTS) 2020 Dataset*. Available at: https://www.kaggle.com/datasets/awsaf49/brats20-dataset-training-validation.

Muhammad Shahid Azeem (2023) *Customer Churn Dataset*. Available at: https://www.kaggle.com/datasets/muhammadshahidazeem/customer-churn-dataset.

Olaf Ronneberger, Philipp Fischer and Thomas Brox (2015) "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*.

5. Appendix: Key Code Snippets

5.1. Task 1: Shallow Neural Network (Pure NumPy)

The following snippets demonstrate the core components of the manually-implemented neural network — forward pass, backpropagation, and the Adam optimiser — all written in pure NumPy without any deep learning frameworks.

```
import numpy as np

def tanh(z):
    return np.tanh(z)

def tanh_derivative(a):
    return 1.0 - a ** 2

def sigmoid(z):
    z = np.clip(z, -500, 500)
    return 1.0 / (1.0 + np.exp(-z))

def binary_cross_entropy(y_true, y_pred, sample_weights=None):
    eps = 1e-12
    y_pred = np.clip(y_pred, eps, 1 - eps)
    loss_per_sample = -(y_true * np.log(y_pred)
                      + (1 - y_true) * np.log(1 - y_pred))
    if sample_weights is not None:
        loss_per_sample *= sample_weights
    return np.mean(loss_per_sample)
```

Listing 1: Activation functions and weighted binary cross-entropy loss implementation.

```python
class ShallowNeuralNetwork:
    def __init__(self, input_size, hidden_size=64, learning_rate=0.005,
                 weight_decay=0.001):
        limit1 = np.sqrt(6.0 / (input_size + hidden_size))
        self.W1 = np.random.uniform(-limit1, limit1,
                                    (input_size, hidden_size))
        self.b1 = np.zeros((1, hidden_size))
        limit2 = np.sqrt(6.0 / (hidden_size + 1))
        self.W2 = np.random.uniform(-limit2, limit2, (hidden_size, 1))
        self.b2 = np.zeros((1, 1))
        self.lr, self.weight_decay = learning_rate, weight_decay

    def forward(self, X):
        self.X = X
        self.Z1 = X @ self.W1 + self.b1
        self.A1 = tanh(self.Z1)
        self.Z2 = self.A1 @ self.W2 + self.b2
        self.A2 = sigmoid(self.Z2)
        return self.A2

    def backward(self, y_true, sample_weights=None):
        m = y_true.shape[0]
        dZ2 = self.A2 - y_true.reshape(-1, 1)
        if sample_weights is not None:
            dZ2 *= sample_weights.reshape(-1, 1)
        dW2 = (self.A1.T @ dZ2) / m + self.weight_decay * self.W2
        db2 = np.mean(dZ2, axis=0, keepdims=True)
        dA1 = dZ2 @ self.W2.T
        dZ1 = dA1 * tanh_derivative(self.A1)
        dW1 = (self.X.T @ dZ1) / m + self.weight_decay * self.W1
        db1 = np.mean(dZ1, axis=0, keepdims=True)
        return {'W1': dW1, 'b1': db1, 'W2': dW2, 'b2': db2}
```

Listing 2: Network architecture with Xavier/Glorot initialisation, manual forward pass, and back-propagation through a single hidden layer.

```python
    def _adam_update(self, grads):
        self.t += 1
        for param in ['W1', 'b1', 'W2', 'b2']:
            g = grads[param]
            self.m[param] = (self.beta1 * self.m[param]
                             + (1 - self.beta1) * g)
            self.v[param] = (self.beta2 * self.v[param]
                             + (1 - self.beta2) * g ** 2)
            m_hat = self.m[param] / (1 - self.beta1 ** self.t)
            v_hat = self.v[param] / (1 - self.beta2 ** self.t)
            update = self.lr * m_hat / (np.sqrt(v_hat) + self.adam_eps)
            setattr(self, param, getattr(self, param) - update)
```

Listing 3: Adam optimiser implemented from scratch with momentum ($\beta_1 = 0.9$), RMSProp ($\beta_2 = 0.999$), and bias-corrected moment estimates.

5.2. Task 2: 3D U-Net for Brain Tumor Segmentation (PyTorch)

The following snippets show the custom 3D U-Net architecture implementing a standard encoder-decoder with skip connections, a combined Dice-Focal loss function for severe class imbalance, and the 2D-to-3D weight inflation transfer learning strategy.

```
class Custom3DUNet(nn.Module):
    def __init__(self, in_channels=4, out_classes=4,
                 init_features=32, dropout=0.3):
        super().__init__()
        self.inc  = DoubleConv3D(in_channels, init_features, dropout)
        self.down1 = Down3D(init_features, init_features * 2, dropout)
        self.down2 = Down3D(init_features * 2, init_features * 4, dropout)
        self.down3 = Down3D(init_features * 4, init_features * 8, dropout)
        self.down4 = Down3D(init_features * 8, init_features * 16, dropout)
        self.up1 = Up3D(init_features * 16, init_features * 8, dropout)
        self.up2 = Up3D(init_features * 8, init_features * 4, dropout)
        self.up3 = Up3D(init_features * 4, init_features * 2, dropout)
        self.up4 = Up3D(init_features * 2, init_features, dropout)
        self.outc = OutConv3D(init_features, out_classes)

    def forward(self, x):
        x1 = self.inc(x)
        x2 = self.down1(x1)
        x3 = self.down2(x2)
        x4 = self.down3(x3)
        x5 = self.down4(x4)
        x = self.up1(x5, x4)
        x = self.up2(x, x3)
        x = self.up3(x, x2)
        x = self.up4(x, x1)
        return self.outc(x)
```

Listing 4: Custom 3D U-Net architecture with symmetric encoder-decoder topology. Down3D applies MaxPool3d followed by DoubleConv3D (two Conv3d + BatchNorm3d + LeakyReLU blocks). Up3D applies ConvTranspose3d and concatenates skip connections from the encoder.

```python
class DiceFocalLoss(nn.Module):
    def __init__(self, lambda_dice=1.0, lambda_focal=1.0):
        super().__init__()
        self.dice = DiceLoss()
        alpha = torch.tensor([0.1, 1.0, 1.0, 1.0])
        self.focal = FocalLoss(alpha=alpha, gamma=2.0)
        self.lambda_dice = lambda_dice
        self.lambda_focal = lambda_focal

    def forward(self, logits, targets):
        dice_loss = self.dice(logits, targets)
        focal_loss = self.focal(logits, targets)
        return self.lambda_dice * dice_loss   \
             + self.lambda_focal * focal_loss

class DiceLoss(nn.Module):
    def forward(self, logits, targets):
        probs = torch.softmax(logits, dim=1)
        targets_oh = torch.zeros_like(probs)
        targets_oh.scatter_(1, targets.long(), 1)
        dims = (2, 3, 4)
        intersection = torch.sum(probs * targets_oh, dim=dims)
        union = (torch.sum(probs, dim=dims)
               + torch.sum(targets_oh, dim=dims))
        dice = (2.0 * intersection + 1e-5) / (union + 1e-5)
        return 1.0 - torch.mean(dice)

class FocalLoss(nn.Module):
    def forward(self, logits, targets):
        ce = F.cross_entropy(logits, targets.squeeze(1).long(),
                             reduction='none')
        pt = torch.exp(-ce)
        return torch.mean(((1 - pt) ** 2.0) * ce)
```

Listing 5: Combined Dice-Focal loss. Dice loss directly optimises spatial overlap regardless of class size, while Focal loss ($\gamma = 2$) down-weights easily classified background voxels (alpha=0.1) and concentrates on hard tumor boundary predictions.

```python
def inflate_2d_to_3d_weights(model_3d):
    resnet2d = _build_resnet18_manually()
    conv2d_layers = [m for m in resnet2d.modules()
                     if isinstance(m, nn.Conv2d)]
    conv3d_layers = [m for m in model_3d.modules()
                     if isinstance(m, nn.Conv3d)]
    inflated = 0
    for c2, c3 in zip(conv2d_layers, conv3d_layers):
        if (c2.in_channels == c3.in_channels
            and c2.out_channels == c3.out_channels
            and c2.kernel_size == (3, 3)
            and c3.kernel_size == (3, 3, 3)):
            w2 = c2.weight.data
            w3 = w2.unsqueeze(2).repeat(1, 1, 3, 1, 1) / 3.0
            c3.weight.data = w3
            inflated += 1
    return model_3d
```

Listing 6: 2D-to-3D weight inflation. Pre-trained ImageNet 2D convolution filters ([Out, In, 3, 3]) are replicated across the depth axis to create 3D kernels ([Out, In, 3, 3, 3]), divided by the depth size to preserve activation variance, and injected into the 3D U-Net encoder.